PRACTICAL 14

Admin Dashboard, Role-Based Access Control (RBAC) and Audit Logs

Student Name	KIRTAN KAKADIYA
Enrollment / Roll No.	25CS026
Program / Semester	3RD
Course / Subject	WEB DEVELOPMENT FRAMEWORK
Faculty Name	BRINDA MAM
Submission Date	31-10-2026

1. Aim

To build a Bootstrap-based Admin dashboard that shows live database statistics and the latest event registrations, to restrict access with role-based access control (RBAC) using role-specific navigation and Admin-only routes, and to record successful Admin actions in a persistent audit log.

2. Practical Requirements

- **Dynamic Dashboard** – Display user, student and event counts and the latest event registrations, read live from the database.
- **Role-Based Navigation** – Show a different menu to Admin and Student users.
- **RBAC & Security** – Block non-admin users from Admin-only pages with an HTTP 403 Unauthorized Access page.
- **Audit Logs** – Record Admin actions (who, what, which entity, when) in an audit_logs table and show them in an Admin-only view.

3. Files Submitted

File	Purpose
`dashboard.php`	Bootstrap dashboard: live statistics, latest registrations and role-based menu
`auth_helpers.php`	Session handling, RBAC check (requireUserRole) and audit helper (recordAuditLog)
`audit_logs.php`	Admin-only page listing recorded audit entries
`schema.sql`	MySQL schema including the audit_logs table, indexes and foreign key

4. Dashboard UI Screenshots

4.1 Admin dashboard – live statistics and latest registrations

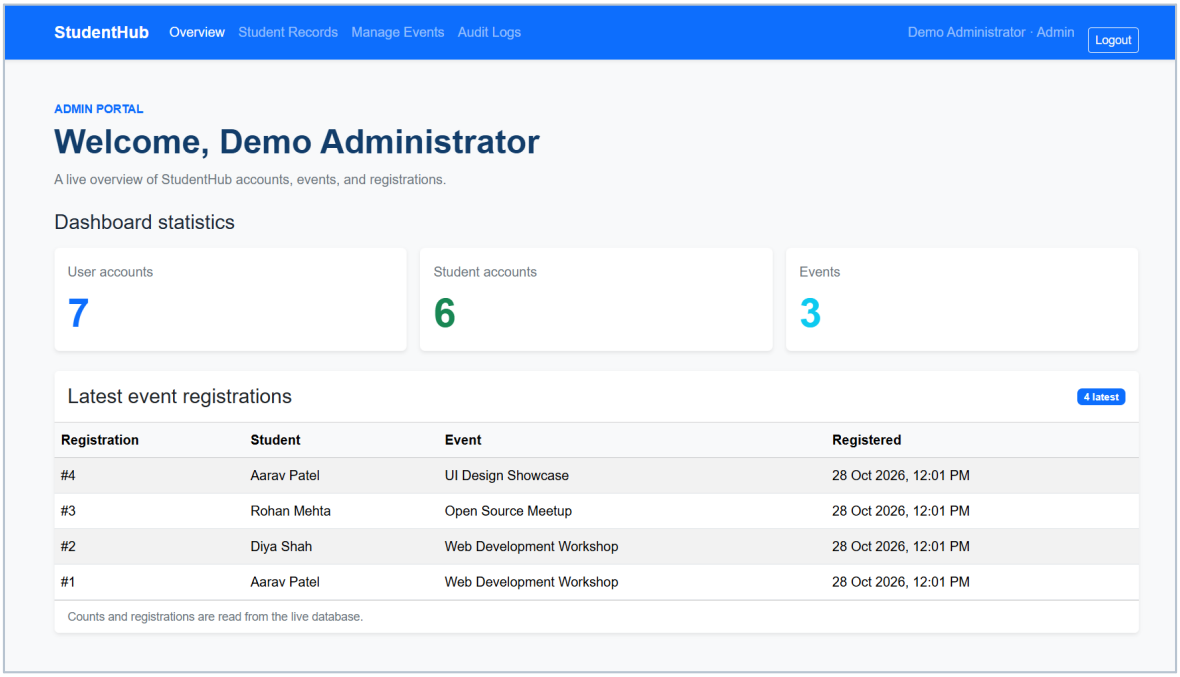


Figure 4.1: Admin dashboard – live statistics and latest registrations

Dashboard item	Value shown
User accounts	7
Student accounts	6
Events	3
Latest event registrations	4 (#4, #3, #2, #1)

The Admin navigation shows **Overview, Student Records, Manage Events** and **Audit Logs**. The displayed counts were compared with direct database queries and matched.

5. RBAC & Security Screenshots

5.1 Student dashboard – role-specific menu (Overview, Student Home, Events)

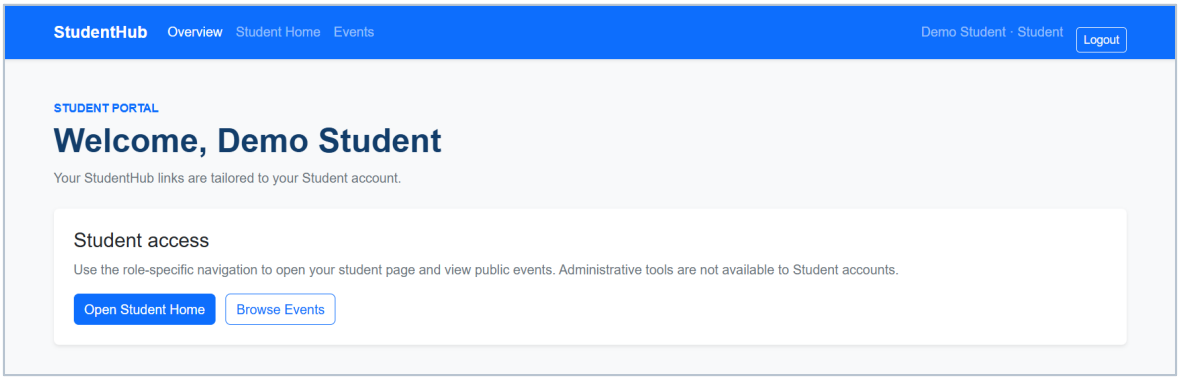


Figure 5.1: Student dashboard – role-specific menu (Overview, Student Home, Events)

5.2 Non-admin access restricted – 403 Unauthorized Access page

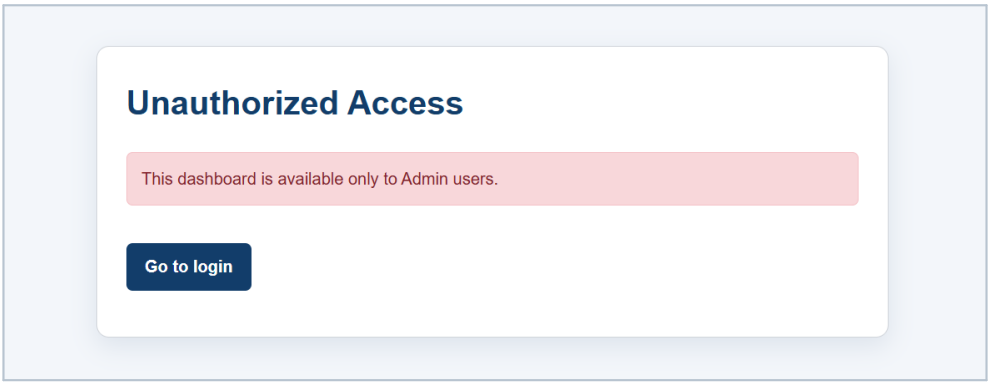


Figure 5.2: Non-admin access restricted – 403 Unauthorized Access page

Admin-only route requested by Student	Result
`admin_dashboard.php`	HTTP 403
`event_management.php`	HTTP 403
`audit_logs.php`	HTTP 403

6. Audit Log Proof

6.1 Audit log table – recorded Admin actions

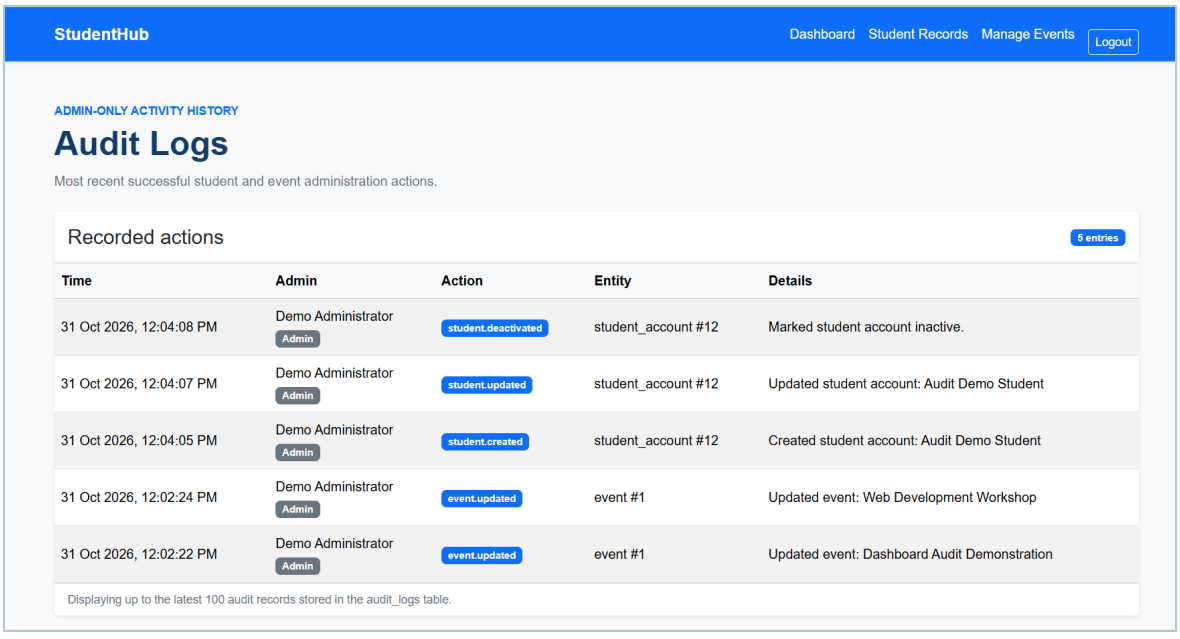


Figure 6.1: Audit log table – recorded Admin actions

The table is populated from the **audit_logs** database table and shows the acting Admin, action, entity, time and details. Visible entries are **student.created**, **student.updated** and **student.deactivated** for student account #12, and two **event.updated** entries for event #1. Audit writes share a database transaction with the corresponding successful student or event change. The test student account was removed after testing, and the audit history was retained as evidence.

7. Code Listings (Key Excerpts)

Key parts of the code are shown below. The complete files are submitted separately.

7.1 dashboard.php – RBAC check, dashboard queries and role-based menu

```
<?php
require __DIR__ . '/auth_helpers.php';
requireAuthenticatedUser();

$role = $_SESSION['role'];
if (!in_array($role, ['Admin', 'Student'], true)) {
    http_response_code(403);
    renderAuthPage('Unauthorized Access', 'Your account does not have a supported dashboard role.', false);
    exit;
}
$dashboardError = '';
$stats = null;
$latestRegistrations = [];

try {
    require __DIR__ . '/db.php';

    if ($role === 'Admin') {
        $statsResult = $mysqli->query(
            'SELECT
                (SELECT COUNT(*) FROM student_accounts) AS user_count,
                (SELECT COUNT(*) FROM student_accounts WHERE role = \'Student\') AS student_count,
                (SELECT COUNT(*) FROM events) AS event_count'
        );
        $stats = $statsResult->fetch_assoc();
        $statsResult->free();

        $registrations = $mysqli->query(
            'SELECT r.registration_id, s.full_name AS student_name, e.title AS event_title, r.registered_at
             FROM registrations r
             INNER JOIN students s ON s.student_id = r.student_id
             INNER JOIN events e ON e.event_id = r.event_id
             ORDER BY r.registered_at DESC, r.registration_id DESC
             LIMIT 6'
        );
        $latestRegistrations = $registrations->fetch_all(MYSQLI_ASSOC);
        $registrations->free();
    }
} catch (RuntimeException | mysqli_sql_exception $exception) {
    error_log('StudentHub dashboard query failed: ' . $exception->getMessage());
    http_response_code(500);
    $dashboardError = 'Dashboard data is temporarily unavailable. Check the database connection and try again.';
}

$adminMenu = [
    ['label' => 'Overview', 'href' => 'dashboard.php'],
    ['label' => 'Student Records', 'href' => 'admin_dashboard.php'],
    ['label' => 'Manage Events', 'href' => 'event_management.php'],
    ['label' => 'Audit Logs', 'href' => 'audit_logs.php'],
];
$studentMenu = [
    ['label' => 'Overview', 'href' => 'dashboard.php'],
    ['label' => 'Student Home', 'href' => 'student_dashboard.php'],
    ['label' => 'Events', 'href' => 'pages/events.html'],
];
$menu = $role === 'Admin' ? $adminMenu : $studentMenu;
```

7.2 auth_helpers.php – requireUserRole() and recordAuditLog()

```
function requireUserRole($requiredRole)
{
    requireAuthenticatedUser();

    if ($_SESSION['role'] !== $requiredRole) {
        http_response_code(403);
        renderAuthPage(
            'Unauthorized Access',
            'This dashboard is available only to ' . $requiredRole . ' users.',
            false
        );
        exit;
    }
}

function recordAuditLog(mysqli $mysqli, string $action, string $entityType, ?int $entityId, string $details): void
{
    $actorId = isset($_SESSION['user_id']) ? (int) $_SESSION['user_id'] : null;
    $actorName = isset($_SESSION['full_name']) ? (string) $_SESSION['full_name'] : 'Unknown user';
    $actorRole = isset($_SESSION['role']) ? (string) $_SESSION['role'] : 'Unknown';
    $statement = $mysqli->prepare(
        'INSERT INTO audit_logs
            (actor_account_id, actor_name, actor_role, action, entity_type, entity_id, details)
         VALUES (?, ?, ?, ?, ?, ?, ?)'
    );
    $statement->bind_param(
        'issssis',
        $actorId,
        $actorName,
        $actorRole,
        $action,
        $entityType,
        $entityId,
        $details
    );
    $statement->execute();
    $statement->close();
}
```

7.3 audit_logs.php – Admin-only audit query

```
<?php
require __DIR__ . '/auth_helpers.php';
requireUserRole('Admin');

try {
    require __DIR__ . '/db.php';
    $result = $mysqli->query(
        'SELECT audit_id, actor_name, actor_role, action, entity_type, entity_id, details, created_at
         FROM audit_logs
         ORDER BY created_at DESC, audit_id DESC
         LIMIT 100'
    );
    $auditEntries = $result->fetch_all(MYSQLI_ASSOC);
    $result->free();
} catch (RuntimeException | mysqli_sql_exception $exception) {
    error_log('StudentHub audit log query failed: ' . $exception->getMessage());
    http_response_code(500);
    $auditError = 'Audit history is temporarily unavailable. Check the database connection and schema.';
    $auditEntries = [];
}
```

7.4 schema.sql – registrations and audit_logs tables

```
CREATE TABLE IF NOT EXISTS registrations (
  registration_id INT UNSIGNED AUTO_INCREMENT PRIMARY KEY,
  student_id INT UNSIGNED NOT NULL,
  event_id INT UNSIGNED NOT NULL,
  registered_at TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP,
  CONSTRAINT unique_student_event UNIQUE (student_id, event_id),
  CONSTRAINT fk_registrations_student
    FOREIGN KEY (student_id) REFERENCES students (student_id)
    ON UPDATE CASCADE ON DELETE CASCADE,
  CONSTRAINT fk_registrations_event
    FOREIGN KEY (event_id) REFERENCES events (event_id)
    ON UPDATE CASCADE ON DELETE CASCADE
) ENGINE=InnoDB;

CREATE TABLE IF NOT EXISTS audit_logs (
  audit_id BIGINT UNSIGNED AUTO_INCREMENT PRIMARY KEY,
  actor_account_id INT UNSIGNED NULL,
  actor_name VARCHAR(100) NOT NULL,
  actor_role VARCHAR(30) NOT NULL,
  action VARCHAR(100) NOT NULL,
  entity_type VARCHAR(64) NOT NULL,
  entity_id INT UNSIGNED NULL,
  details VARCHAR(500) NOT NULL,
  created_at TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP,
  INDEX idx_audit_logs_created_at (created_at),
  INDEX idx_audit_logs_actor (actor_account_id),
  CONSTRAINT fk_audit_logs_actor
    FOREIGN KEY (actor_account_id) REFERENCES student_accounts (account_id)
    ON UPDATE CASCADE ON DELETE SET NULL
) ENGINE=InnoDB;
```

8. Verification

- Imported **schema.sql** and verified the audit_logs table, indexes and account foreign key.
- PHP syntax checks passed for dashboard.php, audit_logs.php, auth_helpers.php, admin_dashboard.php and event_management.php.
- Compared dashboard counts with database queries; the displayed values matched.
- Confirmed Student requests to all three Admin-only routes returned HTTP 403.
- Confirmed successful student and event actions produced corresponding audit rows, including the Admin actor and entity details.

9. Result

A Bootstrap-based Admin dashboard was implemented and tested. It shows live database statistics (7 user accounts, 6 student accounts and 3 events) and the latest event registrations, provides role-specific navigation for Admin and Student users, restricts Admin-only routes with an HTTP 403 page, and keeps a persistent audit trail of Admin actions.

10. Conclusion

This practical showed how to combine live dashboard data, role-based access control and audit logging in a PHP and MySQL application. Role checks on the server keep restricted pages away from non-admin users, while the audit log provides accountability by recording who performed each administrative action, on which entity and when.

— End of Practical 14 Submission —